Task B

Efficiency & System Optimization Plan

Cross-Lingual Information Extraction and Machine Translation System

NLP Applications (S2-25_AIMLCZG519) — Assignment 2, Part 1, Task B (2 Marks)

BITS Pilani Work Integrated Learning Programmes — M.Tech. Artificial Intelligence and Machine Learning

GitHub Repository: https://github.com/hamza-aziz-ai/CrossLingual-IE-MT

Student ID	Name	Contribution
2024AC05133	Hamza Aziz	100%
2024AC05114	Kalpana S.	100%
2024AC05136	Hardik Sharma	100%
2024AC05128	Shubham Srivastava	100%
2024AC05109	Nitin Verma	100%

1. Pipeline Strategy: Translate-then-Extract vs. Direct Cross-Lingual Extraction

1.1 Two Candidate Architectures

Given a text in an Indic language (or English) from which Named Entities, Relations and Events must be extracted, there are two fundamentally different pipeline designs:

Architecture A — Translate-then-Extract (implemented in Part 1)

```
Indic text --[MT: Indic→English]--> English text --[English NER/RE/Event tools]--> IE output
```

Text is first machine-translated into English, and every IE model then runs on the English translation. This is the architecture implemented for this assignment (backend/pipeline.py).

Architecture B — Direct Cross-Lingual Extraction

```
Indic text --[Native Indic-language NER/RE/Event tools]--> IE output   (no translation step)
```

IE models are applied natively to the source-language text, either using models trained directly on that language (e.g. IndicNER for NER) or multilingual/cross-lingual models capable of zero-shot transfer (e.g. an XLM-R-based relation extractor fine-tuned only on English data but applied to Hindi input, relying on shared multilingual embeddings).

1.2 Comparison

Dimension	A: Translate-then-Extract	B: Direct Cross-Lingual Extraction
Error propagation	Translation errors (mistranslated entities, dropped words, ambiguous words resolved incorrectly) propagate directly into every downstream IE result — a mistranslated name becomes an unrecoverable NER miss.	No translation step, so no translation-induced error; IE errors come only from the (typically weaker) cross-lingual/native model itself.
Translation quality dependency	IE quality is upper-bounded by MT quality. A highly fluent but subtly wrong translation (e.g. Hindi कल/"kal" — ambiguous between "yesterday" and "tomorrow" — resolved to the wrong sense) silently corrupts entity/event timing even though the sentence reads fine.	Not applicable in the same way, but native/cross-lingual models still make their own language-specific errors (e.g. weaker performance on morphologically rich Indic verb forms).
Model availability per language	Only needs strong tooling in ONE pivot language (English) — the most resource-rich NLP language in the world. Every IE sub-task reuses the same mature English tooling regardless of source language.	Needs either (a) a dedicated model per language per IE task, which does not exist for Relation Extraction or Event Extraction in Hindi/Tamil/Marathi/Bengali today, or (b) a genuinely multilingual/cross-lingual model, which is rarer and generally lower-accuracy than an English-only counterpart for RE/Event tasks specifically.
Engineering complexity	One extra network hop (translation) per request, but a single, uniform IE code path regardless of input language.	Simpler data flow (no translation step) but branching IE logic per language, and a much larger model/testing surface if per-language models are used.
Latency	Adds full MT inference latency before IE can even start — the two stages are sequential, not parallel. ≈40-100s/sentence on the CPU-only development machine, ≈0.9s on the faster machine used for screenshot capture (see the hardware note in Section 2).	Only IE inference latency; no MT stage. Lower latency in principle if a native/cross-lingual model of comparable quality exists.
Observed in this implementation	Fully implemented for NER, RE and Event/Temporal (Section 2.2 of System Report).	Implemented for NER only (IndicNER runs natively on Hindi/Tamil/Marathi/Bengali text); RE and Event Extraction are English-pivoted only — see Limitations below.

1.3 Why Translate-then-Extract Was Chosen for This Implementation

- Relation Extraction: no freely available, pip-installable Relation Extraction model exists for Hindi/Tamil/Marathi/Bengali with anywhere near REBEL's ~220-relation-type coverage for English. Building one would require either substantial fine-tuning data collection (out of scope for this assignment) or accepting much lower recall from a zero-shot cross-lingual transfer model.
- Event/Temporal Extraction: dependency-parse-based event trigger detection and dateparser's temporal grammar are both English-tuned; equivalent quality dependency parsers and locale-aware date grammars are not uniformly available as pip packages across all four target languages.
- Named Entity Recognition is the one sub-task where a genuinely native option (IndicNER) exists and is implemented and demonstrated in this submission — the architecture is not "all-or-nothing"; it uses Direct Extraction where good tooling exists and pivots through English where it does not.

1.4 Quantitative Evidence of the Trade-off

During development, a Hindi sentence translated by NLLB-200 produced the correct English sentence “Tim Cook announced the new iPhone yesterday in California.” from the Hindi source containing “कल” (kal). Because “kal” is genuinely ambiguous between “yesterday” and “tomorrow” in Hindi without broader discourse context, a lower-quality translation model could just as easily have produced “tomorrow” here — which would silently invert the Event/Temporal module's `normalized_time` output (a nearly 24-hour date error) despite the rest of the pipeline reporting no error. This is a concrete illustration of Architecture A's central risk: IE correctness for Indic-language input is only as good as one upstream MT decision per ambiguous word, and that decision is invisible to the IE modules downstream. Direct Cross-Lingual Extraction (Architecture B) would not eliminate temporal ambiguity in the source language, but it would at least keep the ambiguity resolution and the temporal extraction in the same language/model, rather than splitting them across two independent models that cannot see each other's uncertainty.

1.5 Recommendation

For a production system, a hybrid strategy is recommended: keep Translate-then-Extract as the default (it is implemented, functional, and reuses mature tooling), but (a) expose the MT model's own confidence/entropy on ambiguous temporal and named-entity tokens to flag low-confidence translations for human review, and (b) invest in Direct Extraction specifically for NER — already the cheapest sub-task to do natively (Section 1.3) — since it removes the single highest-value category of translation-induced entity errors (misspelled/mistranslated proper nouns) without needing new RE/Event models.

2. Scalability & Inference Optimization

The current implementation prioritizes correctness and tooling maturity over latency (Section 1), and consequently loads full-precision (fp32) models: NLLB-200-distilled-600M (≈2.4 GB), IndicNER (≈1.1 GB) and REBEL-large (≈1.6 GB). Measured end-to-end pipeline latency on the CPU-only machine used for early development was ≈40-100 seconds per sentence for translation alone (Section 7, System Report), which is unacceptable for a low-latency interactive web product. This section proposes concrete model-compression techniques to close that gap.

Note on the two latency figures in this submission: the ≈40-100s/sentence range above and the sub-second per-module timings captured live in Figure 8 of the System Report (Translation ≈0.87s, full pipeline ≈1.36s) are both genuine measurements — they were taken on different machines. The CPU-only figure reflects the constrained development environment used for early testing and is retained here as the conservative baseline this section's optimization proposals are designed against; Figure 8 was captured later, during the screenshot-capture pass described in System Report Section 5.6, on hardware with materially faster inference (GPU-accelerated and/or a substantially faster CPU). The order-of-magnitude optimization argument in this section holds either way: quantization, pruning, and distillation are still the correct levers to pull to make worst-case CPU-only latency acceptable, even though best-case hardware already performs well.

2.1 Quantization

Quantization reduces the numerical precision of model weights (and optionally activations) from 32-bit floating point to a lower-precision representation, shrinking memory footprint and increasing throughput (lower-precision arithmetic is faster on both CPU SIMD units and GPU tensor cores).

- Dynamic INT8 quantization (torch.quantization.quantize_dynamic) — applied post-training, no calibration dataset needed; typically yields ~2–4× CPU inference speedup and ~4× memory reduction for linear-layer-dominated Transformer models like NLLB and REBEL, with commonly reported quality loss under 1 BLEU point / 1 F1 point for this class of model.
- 8-bit / 4-bit weight loading (bitsandbytes, load_in_8bit=True / load_in_4bit=True) — loads the ~2.4 GB NLLB checkpoint in under 1 GB of memory with minimal code change (a single argument to from_pretrained), the single highest-leverage change available given the current implementation already uses HuggingFace Transformers.

- ONNX Runtime / OpenVINO export with INT8 static quantization — requires a short calibration pass over representative sentences but typically outperforms dynamic quantization on CPU, and is a natural next step once a representative traffic sample exists in production.

2.2 Pruning

Pruning removes redundant weights or entire structural units (attention heads, feed-forward neurons, whole layers) that contribute little to model output.

- Magnitude-based unstructured pruning — zero out the smallest-magnitude weights (e.g. bottom 30–40%) and fine-tune briefly to recover quality; reduces stored model size but requires sparse-aware inference kernels to realize a real speedup, which are not yet universally available for Transformer encoder-decoders.
- Structured pruning (attention-head pruning, layer dropping) — more directly translates to real latency reduction on standard hardware since it removes whole matrix-multiply operations rather than individual weights. NLLB-200-distilled-600M already has fewer layers than the 3.3B teacher model (see distillation, Section 2.3); a further-pruned variant (e.g. dropping the deepest 2–4 decoder layers) is a reasonable next iteration if a small further BLEU drop is acceptable for the latency win.
- Early-exit / layer-skipping at inference time — for high-confidence, short, unambiguous sentences (the majority of demo-style input in this application), a shallower forward pass could suffice; this requires a confidence-estimation head and is a longer-term research direction rather than a near-term fix.

2.3 Knowledge Distillation

Knowledge distillation trains a smaller "student" model to reproduce a larger "teacher" model's output distribution, typically recovering most of the teacher's quality at a fraction of the parameter count and inference cost.

- The MT model already benefits from this: facebook/nllb-200-distilled-600M is itself a distilled student of the 3.3B-parameter NLLB-200 teacher, which is precisely why it was chosen over the full-size model for this assignment (System Report, Section 2.3) — distillation is not merely a proposal here, it is already load-bearing in the current design.
- Further distillation to a smaller NLLB checkpoint (e.g. an unofficial ~200–300M-parameter distillation, or task-specific distillation restricted to just the 4 language pairs actually needed — English↔Hindi/Tamil/Marathi/Bengali — rather than all 200 NLLB languages) would shrink both memory footprint and the effective vocabulary/output layer size, which dominates parameter count in massively multilingual MT models.
- Distilling REBEL (currently BART-large-based, ~400M parameters) into a smaller seq2seq student, or replacing it with a lighter classification-based RE model distilled from REBEL's outputs on an in-domain corpus, would address Relation Extraction — currently the single largest optional model in the stack (~1.6 GB) and, per Section 1.3, already the sub-task with weakest cross-lingual options, so a smaller purpose-built model is doubly justified.

2.4 Additional Deployment-Level Optimizations

- Response caching — memoize (translation, entities, relations, events) keyed on a hash of the normalized input text + language pair; repeated demo/test inputs (common in a course-project grading context) become near-instant on cache hit.
- Model warm pools / persistent server process — the current Streamlit app already avoids the worst case (st.cache_resource keeps models resident across requests within one process, Section "Streamlit-specific design notes" of frontend/app.py); a production deployment should additionally keep a small pool of always-warm worker processes to avoid any cold-start latency for the first request after deploy.
- Batching — if concurrent users submit sentences close together in time, batch them into a single forward pass through the MT/NER/RE models rather than serializing requests; the current implementation processes one sentence at a time (backend/mt_engine.py, MTEngine.translate), which is the correct choice for a single-user demo but leaves throughput on the table under concurrent load.

- Right-sized hardware placement — route the 3 large IE-adjacent models (MT, NER, RE) to GPU inference where available (the codebase already auto-selects CUDA when present, backend/mt_engine.py MTEngine._ensure_loaded) and reserve CPU-only serving for the lightweight rule-based fallback path and the Event/Temporal module, which are already fast (≈2–3 seconds, Section 7 System Report) without a GPU.

2.5 Expected Impact (Order-of-Magnitude Estimates)

The figures below are drawn from commonly reported ranges in the model-compression literature for comparably-sized Transformer encoder-decoder / encoder-only models, not from benchmarking performed on this specific system (Section 6, System Report, notes this as a limitation of the current submission).

Technique	Typical memory reduction	Typical latency improvement	Typical quality impact
Dynamic INT8 quantization	~4×	~2–4× (CPU)	<1 BLEU / <1 F1 point
8-bit weight loading (bitsandbytes)	~2–4×	Modest on GPU; enables fitting larger batches	Negligible
Structured pruning (layer/head dropping)	10–30% (depends on depth removed)	Proportional to layers/heads removed	Depends on fine-tuning budget; can be small with recovery fine-tuning
Knowledge distillation (further, beyond NLLB's existing distillation)	2–10× fewer parameters	Proportional to size reduction	Moderate — requires careful student architecture/data choice to keep BLEU/F1 drop small
Response caching	N/A	Near-instant on cache hit	None (exact match only)

3. Conclusion

The Translate-then-Extract architecture implemented for Part 1 is the pragmatic choice given current open-source model availability for Relation Extraction and Event/Temporal Extraction in Hindi, Tamil, Marathi and Bengali, but it carries a measurable, illustrated risk of silent error propagation from ambiguous MT decisions (Section 1.4). Named Entity Recognition already demonstrates that Direct Cross-Lingual Extraction is viable where suitable models exist, and is the recommended next investment. On the performance side, quantization (particularly 8-bit weight loading, given its minimal implementation cost against the existing HuggingFace-based codebase) is the highest-leverage near-term optimization, with structured pruning and further task-specific distillation as follow-on work once a production traffic profile is available to guide where to spend a fine-tuning/recovery budget.